

Jeux vidéo et visée automatique : Entre innovation et contrôle

REGRAGUI ILYASS

CPGE AL ZAHRAWI

10 JUIN 2024



1 Introduction

- Objectif
- Problème
- Solution

2 Squelletisation

- Onde de contour
- Équation de l'onde
- Formalisation et optique
- Retour sur l'onde de contour

3 ALGO ANTI-TRCIHE

4 Dispositif de visée automatique



Objectif

Permettre au joueur de viser efficacement la tête de l'ennemi pour infliger plus de dégâts.



Figure: Counter-Strike 2 - Entraînement avec bots - Match à mort

Problème



Figure: Joueur de Counter-Strike que l'on veut viser à la tête

Possibilité d'utiliser le
deep learning et les
réseaux (RNN)



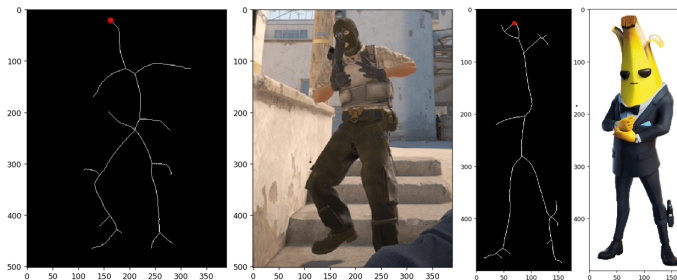
Figure: Joueur de Fortnite avec des formes étranges.

Limites des techniques actuelles
face à des formes non
conventionnelles.



Solution

Réduction des objets à leur squelette médian afin de cibler précisément le point rouge.



Comment réaliser cette solution ?

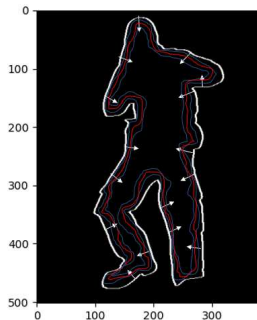


Onde de contour

Nous simulons le contour de l'objet comme une onde se propageant avec une vitesse $\vec{F}$ vers l'intérieur :

Soit $W(t)$ la frontière en mouvement (front) de l'objet, $\vec{n}$ la normale dirigée vers l'intérieur de W . Le mouvement du front est donné par :

$$\frac{\partial W}{\partial t} = \vec{F} \vec{n}$$



Évolution de l'onde de contour.



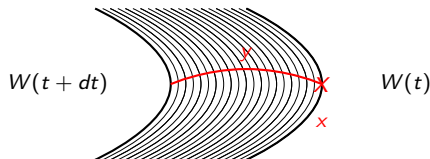
Équation de l'onde

Soit $T(x)$ le temps minimal pour que le point x soit atteint par le front :

$$T(x) := \min\{t \geq 0 \mid x \in W(t)\} \quad \forall x \in \mathbb{R}^2,$$

alors le front $W(t)$ à l'instant t est donné par :

$$W(t) = \{x \mid T(x) = t\}.$$

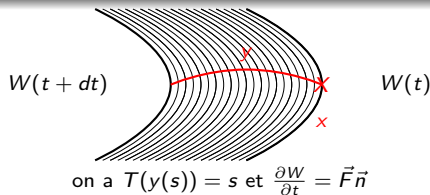


Prenons $x \in W(t)$, et considérons le chemin $y(s)$ issu de x et tel que $y(s) \in W(t)$, pour un voisinage de t . Donc :

$$T(y(s)) = s$$

pour tout s voisin de t .





En prenant la dérivée totale de T par rapport à s :

$$\frac{d}{ds} T(y(s)) = 1$$

$$\nabla T \cdot \frac{\partial y}{\partial s} = 1$$

$$\nabla T \cdot (F \vec{n}) = 1$$

$$\|\nabla T\|^2 = \frac{1}{F^2}.$$

L'équation $\|\nabla T\|^2 = \frac{1}{F^2}$ est une forme bien connue de l'équation d'éikonale, un concept clé de l'optique géométrique.



Définition du Lagrangien

Le lagrangien est une fonction des variables dynamiques d'un système qui permet d'écrire les équations du mouvement du système. En mécanique newtonienne (classique) : le lagrangien $\mathcal{L}$ est souvent défini comme la différence entre l'énergie cinétique T et l'énergie potentielle V :

$$\mathcal{L} = T - V$$

Action S et principe de moindre action

Entre deux instants t_1 et t_2 , le mouvement d'un système matériel est celui qui réalise une valeur stationnaire de l'action S :

$$S_{q_0, t_0}(q, t) = \int_{\gamma} \mathcal{L} dt.$$

γ : une courbe extrémale reliant les points (q_0, t_0) et (q, t)

$\mathcal{L}(q, \dot{q})$: le lagrangien.

En d'autres termes, parmi tous les chemins possibles reliant (q_0, t_0) et (q, t) , la trajectoire γ suivie par la particule est celle qui minimise la fonction d'action.



Théorème (Équation d'Euler-Lagrange)

La courbe γ est une extrémale de la fonctionnelle

$$\psi(\gamma) = \int_{t_0}^{t_1} \mathcal{L}(q, \dot{q}, t) dt$$

dans l'espace des courbes joignant (q_0, t_0) et (q_1, t_1) , si et seulement si les équations d'Euler-Lagrange suivantes

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}} - \frac{\partial \mathcal{L}}{\partial q} = 0$$

sont satisfaites le long de la courbe γ .



Moment conjugué et Hamiltonien

Moment conjugué

Le moment conjugué p est défini comme :

$$p = \frac{\partial L}{\partial \dot{q}}$$

où L est le lagrangien du système, q représente les coordonnées généralisées et $\dot{q}$ leurs vitesses correspondantes.

Hamiltonien

Le Hamiltonien H est défini par :

$$H = p\dot{q} - L$$

où L est le lagrangien du système.

Équations de Hamilton

Les équations de Hamilton sont données par :

$$\dot{q} = \frac{\partial H}{\partial p}, \quad \dot{p} = -\frac{\partial H}{\partial q}$$



Principe variationnel et optique

Par le principe de Fermat en optique, la trajectoire d'un rayon $\gamma(t) = (x(t), y(t))$ reliant deux points $\gamma(t_0) = \mathbf{q}_0$ et $\gamma(t_1) = \mathbf{q}_1$ minimise l'intégrale suivante :

$$\psi(\gamma) = \int_{\mathbf{q}_0}^{\mathbf{q}_1} n \, ds = \int_{y_0}^{y_1} n \sqrt{1 + \left(\frac{dx}{dy}\right)^2} dy = \int_{y_0}^{y_1} \mathcal{L}(x, x') dy$$

pour

$$\phi(x, y) = \min_{\gamma} (\psi(\gamma)).$$

- $\psi(\gamma)$: L'intégrale à minimiser, représentant le chemin optique
- n : L'indice de réfraction du milieu, qui peut varier en fonction de la position.
- ds : Un élément infinitésimal de longueur de la trajectoire γ .

On peut démontrer que (voir annexe) :

$$\|\nabla \phi\|^2 = n^2$$

Exemple : dans un milieu pour lequel $n = 1/f$ on a

$$(\|\nabla \phi\| f)^2 = 1,$$



- Puisque l'onde obéit à la même équation que celle suivie d'un rayon lumineux donc :
on peut considérer que l'action de système (coût à minimiser) est égale à la longueur optique entre un point q_0 et q

$$S_{q_0, t_0}(q, t) = \int_{\gamma} L dt$$

- Donc

$$L = \frac{1}{F(x, y)} \left\| \frac{\partial \gamma}{\partial t} \right\| = \frac{1}{F(x, y)} \|\dot{q}\| \quad L \text{ le lagrangien de système}$$

- Et (équation eikonal) :

$$(\nabla S f)^2 = 1$$



Cas où $f = 1$

- On obtient

$$(\nabla S)^2 = 1$$

- Et

$$S_{q_0, t_0}(q, t) = \int_{\gamma} \|\dot{q}(t)\| dt$$

Si la trajectoire γ est une ligne droite, cette intégrale correspond à la distance euclidienne entre q_0 et q :

$$S_{q_0, t_0}(q, t) = \int_{\gamma} \|\dot{q}(t)\| dt = \|q - q_0\| = d(q, q_0)$$

d'où l'action devient égale à la distance euclidienne.



Système Hamiltonien

- On a le Hamiltonien

$$H = \mathbf{p} \cdot \dot{\mathbf{q}} - L$$

$$H(q, p, t) = \mathbf{p} \cdot \dot{\mathbf{q}} - \|\dot{\mathbf{q}}\| = 1 - \|\nabla D\|$$

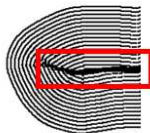
- et le système Hamiltonien est donné par

$$\begin{cases} \dot{\mathbf{p}} = -\frac{\partial H}{\partial \mathbf{q}} = (0, 0), \\ \dot{\mathbf{q}} = \frac{\partial H}{\partial \mathbf{p}} = -\nabla D \end{cases}$$



Théorème 1

L'énergie totale $H(p, q)$ du système hamiltonien reste constante le long des trajectoires.



Chocs → La loi de conservation de l'énergie est enfreinte.

La clé est de mesurer le flux du champ vectoriel ($\dot{q}$), qui est analogue au flux d'un fluide incompressible tel que l'eau. Notez que pour un volume avec une surface fermée, un excès de sortie ou d'entrée de flux à travers la surface indique la présence d'une source ou d'un puits, respectivement, dans le volume.



On peut mesurer le flux moyen en un point (facile à calculer en traitement d'image) : défini comme le rapport entre le flux de sortie d'une région R contenant ce point et le périmètre de cette région :

$$\frac{\int_{\partial R} \langle \dot{\mathbf{q}}, \mathbf{N} \rangle ds}{\mathcal{P}(R)},$$

où $\langle \cdot, \cdot \rangle$ est le produit scalaire, ds est une portion du contour de R et $\mathbf{N}$ est la normale de sortie en chaque point du contour.

d'où l'algorithme suivant :



Données : Objet Ω .

Résultat : Carte du Flux Sortant Moyen.

- ① Calculer la transformation de distance euclidienne numérique ϕ de l'objet (Borgefors, 1984) ;
- ② Calculer le champ vectoriel gradient $\nabla\phi$;
- ③ Calculer le flux sortant moyen de $\nabla\phi$:
 - Pour chaque point x faire

$$F(x) = -\frac{1}{n} \sum_{i=1}^n \langle \mathbf{n}_i, \nabla\phi(x_i) \rangle;$$

où x_i est un des n voisins de x ($n = 8$ en 2D, $n = 26$ en 3D) et $\mathbf{n}_i$ est la normale sortante en x_i de la sphère unitaire centrée en x .



Code Python

```
1 def sample_sphere_2D(number_of_samples):
2     sphere_points = np.zeros((number_of_samples, 2))
3     alpha = (2 * math.pi) / number_of_samples
4     for i in range(number_of_samples):
5         sphere_points[i][0] = math.cos(alpha * i)
6         sphere_points[i][1] = math.sin(alpha * i)
7     return sphere_points
8
9 filename = "C:/Users/Ilyas/OneDrive/Bureau/horse.png"
10 I = Image.open(filename)
11 I = np.array(I)
12
13 I = (I * 255).astype(np.uint8)
14 I = rgb2gray(I)
15 I = 1 - I
16
17 I_image = Image.fromarray((I * 255).astype(np.uint8))
18 I_image_resized = I_image.resize((int(I_image.width * 0.3), int(I_image.height * 0.3)), Image.
    LANCZOS)
19 I_resized = np.array(I_image_resized) / 255.0
20
21 number_of_samples = 60
22 epsilon = 1
23 flux_threshold = 18
24
25 distImage, IDX = distance_transform_edt(I_resized, return_indices=True)
26 sphere_points = sample_sphere_2D(number_of_samples)
```



```
1 def compute_aof(distImage, IDX, sphere_points, epsilon):
2     m = distImage.shape[0]
3     n = distImage.shape[1]
4     normals = np.zeros(sphere_points.shape)
5     fluxImage = np.zeros((m, n))
6     for t in range(0, number_of_samples):
7         normals[t] = sphere_points[t]
8     sphere_points = sphere_points * epsilon
9
10    XInds = IDX[0]
11    YInds = IDX[1]
12
13    for i in range(0, m):
14        for j in range(0, n):
15            flux_value = 0
16            if (distImage[i][j] > -1.5):
17                if (i > epsilon and j > epsilon and i < m - epsilon and j < n - epsilon):
18                    for ind in range(0, number_of_samples):
19                        px = i + sphere_points[ind][0] + 0.5
20                        py = j + sphere_points[ind][1] + 0.5
21                        ci = math.floor(i + sphere_points[ind][0] + 0.5)
22                        cj = math.floor(j + sphere_points[ind][1] + 0.5)
23                        bx = XInds[ci][cj]
24                        by = YInds[ci][cj]
25                        qq = [bx - px, by - py]
26                        d = np.linalg.norm(qq)
27                        if (d != 0):
28                            qq = qq / d
29                        else:
30                            qq = [0, 0]
31                        flux_value = flux_value + np.dot(qq, normals[ind])
32            fluxImage[i][j] = flux_value
33    return fluxImage
```



Code Python

```
1 fluxImage = compute_aof(distImage, IDX, sphere_points, epsilon)
2
3 # Generate skeleton image based on flux threshold
4 skeletonImage = np.zeros_like(fluxImage)
5 skeletonImage[fluxImage > flux_threshold] = 1
6
7 # Display skeleton image
8 plt.imshow(skeletonImage, cmap='gray')
9 plt.axis('off')
10 plt.show()
```





La méthode développée pourrait être utilisée de manière illicite. Malgré les programmes antitriche, les jeux restent vulnérables aux dispositifs externes. Un algorithme probabiliste qui analyse le comportement des joueurs peut résoudre ce problème, prévenant ainsi les utilisations non éthiques de la visée automatique.



HYPOTHESE

- Nous supposons qu'il y a $N \geq 2$ joueurs $i = 1, \dots, N$ participant. Il est important de noter que dans les jeux en ligne, le nombre de joueurs N peut varier considérablement, allant parfois jusqu'à $N \approx 1000$ ou plus. Chaque joueur i affronte tous les autres joueurs $j \neq i$ au moins une fois. Nous notons X_{ij} comme une variable aléatoire représentant le nombre de fois où le joueur i bat le joueur j , avec $X_{ii} = 0$, et $n_{ij} > 0$ représentant le nombre de fois où les joueurs i et j jouent ensemble. (les variables X_{ij} sont indépendantes)
- Chaque joueur i possède un classement π_i de telle manière que $0 \leq \pi_i$. (peut être estimé à partir des résultats des parties)
- Le résultat d'une partie entre les joueurs i et j est défini par une fonction d'issue appelée la fonction $g(\cdot, \cdot)$, avec $0 \leq g(\cdot) \leq 1$. Un joueur i bat un joueur j avec une probabilité de $g(\pi_i, \pi_j) = 1 - g(\pi_j, \pi_i)$.



HYPOTHESE

- Chaque partie de CS:GO implique généralement 8 joueurs (Mode Match à Morts).
- Les résultats des matchs sont enregistrés à l'aide de variables aléatoires X_{ij} , représentant le nombre de fois où le joueur i élimine le joueur j dans une partie. $X_{ii} = 0$, et $n_{ij} > 0$ représente le nombre de fois où les joueurs i et j se sont affrontés.
- Dans une compétition de CS:GO, il peut y avoir un grand nombre de joueurs participant à différents matchs, ce qui rend N élevé.
- Chaque joueur i a un classement π_i (rank) qui mesure son habileté dans CS:GO, basé sur des statistiques de performance.
- La fonction d'issue $g(\cdot, \cdot)$ détermine la probabilité que le joueur i batte le joueur j dans une partie de CS:GO. Elle est modélisée de manière similaire à la formule d'estimation de classement Elo :
$$g(\pi_i, \pi_j) = \frac{1}{1 + 10^{(\pi_j - \pi_i)/400}}.$$



Modèle statistique sans triche

Considérons le cas où il n'y a pas de tricheur. Donc la probabilité que joueur i batte k fois le joueur j suit une loi binomiale :

$$P(X_{ij} = k) = \frac{s_{ij}!}{k! (s_{ij} - k)!} \cdot g(\pi_i, \pi_j)^k \cdot g(\pi_j, \pi_i)^{s_{ij} - k}$$

Soit X_i la variable aléatoire qui représente le nombre total de gains de joueur i :

$$X_i = \sum_{j=1}^N X_{ij}$$

Donc :

$$E(X_i) = E\left(\sum_{j=1}^N X_{ij}\right) = \sum_{j=1}^N E(X_{ij}) = \sum_{j=1}^N s_{ij} g(\pi_i, \pi_j)$$

et

$$\sigma_i^2 = V(X_i) = \sum_{j=1}^N V(X_{ij}) = \sum_{j=1}^N s_{ij} g(\pi_i, \pi_j) (1 - g(\pi_i, \pi_j))$$



- Selon le théorème central limite : (approximativement)

$$\frac{X_i - E(X_i)}{\sigma_i} \sim N(0, 1)$$

- Cela signifie que pour un grand nombre de jeux, X_i suit une distribution normale.
- Pour un seuil ξ donné :

$$X_i \leq E(X_i) + \xi \cdot \sigma_i$$

- Exemple : $\xi = 3$ signifie une confiance de 99.7



Algorithme de détection de triche basé sur les LLNs

Algorithm 1: Algorithme de détection de triche basé sur les LLNs

Data: Rangs réels π_i , $i = 1, \dots, N$. Valeurs calculées $E(X_i)$, σ_i et valeurs observées w_i

repeat

for $i = 1, \dots, N$ **do**

if $w_i > E(X_i) + \xi \cdot \sigma_i$ **then**

 └ le joueur i est détecté comme tricheur

else

 └ le joueur i est détecté comme non-tricheur

until;



Pour tester l'efficacité de ces algorithmes, nous avons développé un dispositif de visée automatique. Ce dispositif est constitué d'un pan-tilt commandé par un Raspberry Pi équipé d'une caméra.



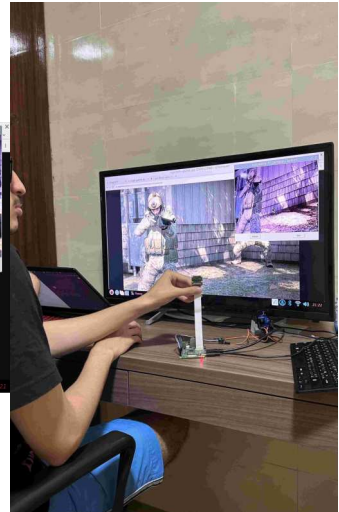
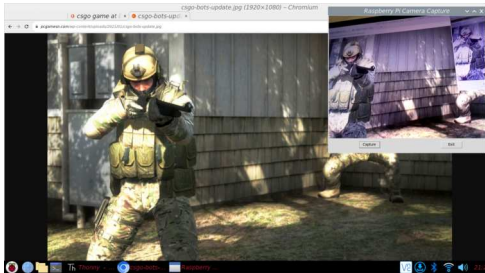
Dispositif



Figure: Raspberry Pi muni d'une caméra.

Figure: Pan-tilt.

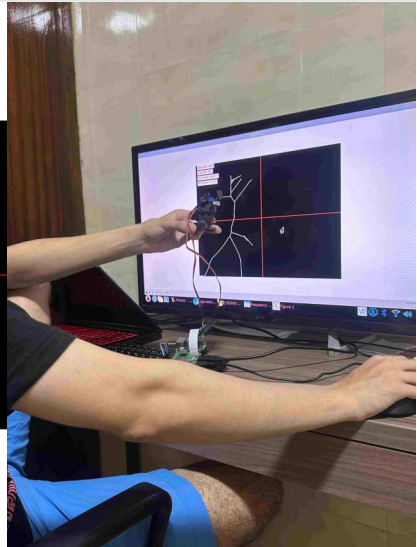
Interface



Dispositif



Figure:



Conclusion

Finalement, durant notre exposé, nous avons réalisé nos principaux objectifs, à savoir :

- Conception d'un algorithme robuste d'extraction du squelette.
- Implémentation d'un dispositif externe capable de cibler automatiquement les joueurs, en utilisant un moteur (pan-tilt) commandé par une carte de commande.
- Développement d'un algorithme probabiliste afin de limiter les cas potentiels de triche, préservant ainsi l'équité et l'intégrité dans l'utilisation de cette technologie.



Annexes



```

import requests
from PIL import Image, ImageTk, ImageChops
import numpy as np
import random
from rembg import remove # Biblioth que pour supprimer les
    arri re-plans
from skimage import img_as_bool, io, color, morphology #
    Biblioth que pour la squelettisation des images
import matplotlib.pyplot as plt # Biblioth que pour tracer des
    graphiques
import math # Biblioth que pour les op rations math matiques
import pigpio
from time import sleep
import tkinter as tk
from tkinter import ttk, messagebox
from picamera import PiCamera
from picamera.array import PiRGBArray
from scipy.ndimage import distance_transform_edt # Biblioth que
    pour la transformation de distance

def obtenir_couleur_diff rente(chemin_image):
    image = Image.open(chemin_image)
    couleurs = image.getcolors(image.size[0] * image.size[1])
    couleurs_uniques = [c[1][:3] for c in couleurs]
    nouvelle_couleur = (random.randint(0, 255), random.randint(0,
        255), random.randint(0, 255))
    while any(abs(nouvelle_couleur[0] - c[0]) < 50 and abs(
        nouvelle_couleur[1] - c[1]) < 50 and abs(nouvelle_couleur
        [2] - c[2]) < 50 for c in couleurs_uniques):
        nouvelle_couleur = (random.randint(0, 255), random.randint
            (0, 255), random.randint(0, 255))
    return nouvelle_couleur

def pretraiter_image(chemin_image, taille_cible):
    image_input = Image.open(chemin_image)
    input_array = np.array(image_input)
    nouvelle_couleur = obtenir_couleur_diff rente(chemin_image)
    foreground = remove(input_array)
    foreground_image = Image.fromarray(foreground)
    background_image = Image.new("RGB", foreground_image.size,
        tuple(nouvelle_couleur))
    output_image = Image.alpha_composite(background_image.convert("
        RGBA"), foreground_image.convert("RGB"))
    output_image = output_image.resize(taille_cible, Image.LANCZOS)
    largeur, hauteur = output_image.size
    for y in range(hauteur):
        for x in range(largeur):
            r, g, b = output_image.getpixel((x, y))
            output_image.putpixel((x, y), (255, 255, 255) if (r, g,
                b) != nouvelle_couleur else (0, 0, 0))
    output_image = output_image.convert("L")
    preprocessed_image = np.array(output_image) / 255.0
    return preprocessed_image

def echantillonner_sphere_2D(nombre_echantillons):
    sphere_points = np.zeros((nombre_echantillons, 2))
    alpha = (2 * math.pi) / nombre_echantillons

```

```

    for i in range(nombre_echantillons):
        sphere_points[i][0] = math.cos(alpha * i)
        sphere_points[i][1] = math.sin(alpha * i)
    return sphere_points

def sub2ind(array_shape, rows, cols):
    ind = rows * array_shape[1] + cols
    ind[ind < 0] = -1
    ind[ind >= array_shape[0] * array_shape[1]] = -1
    return ind

def ind2sub(array_shape, ind):
    ind[ind < 0] = -1
    ind[ind >= array_shape[0] * array_shape[1]] = -1
    rows = (ind.astype('int') // array_shape[1])
    cols = ind % array_shape[1]
    return (rows, cols)

def calculer_aof(distImage, IDX, sphere_points, epsilon):
    m = distImage.shape[0]
    n = distImage.shape[1]
    normals = np.zeros(sphere_points.shape)
    fluxImage = np.zeros((m, n))
    for t in range(nombre_echantillons):
        normals[t] = sphere_points[t]
    sphere_points = sphere_points * epsilon
    XInds = IDX[0]
    YInds = IDX[1]
    for i in range(m):
        for j in range(n):
            flux_value = 0
            if distImage[i][j] > -1.5:
                if i > epsilon and j > epsilon et i < m - epsilon
                et j < n - epsilon:
                    for ind in range(nombre_echantillons):
                        px = i + sphere_points[ind][0] + 0.5
                        py = j + sphere_points[ind][1] + 0.5
                        cI = math.floor(i + sphere_points[ind][0] +
                                0.5)
                        cJ = math.floor(j + sphere_points[ind][1] +
                                0.5)
                        bx = XInds[cI][cJ]
                        by = YInds[cI][cJ]
                        qq = [bx - px, by - py]
                        d = np.linalg.norm(qq)
                        if d != 0:
                            qq = qq / d
                        else:
                            qq = [0, 0]
                        flux_value = flux_value + np.dot(qq,
                                normals[ind])
                    fluxImage[i][j] = flux_value
    return fluxImage

def squelettiser_et_traiter(chemin_image, api_key, taille_cible
=(300, 300), nombre_echantillons=60, epsilon=1, seuil_flux=18):
    preprocessed_image = pretraiter_image(chemin_image,

```

```

        taille_cible)
    distImage, IDX = distance_transform_edt(preprocessed_image,
        return_indices=True)
    sphere_points = echantillonner_sphere_2D(nombre_echantillons)
    fluxImage = calculer_aof(distImage, IDX, sphere_points, epsilon
    )
    skeletonImage = np.zeros_like(fluxImage)
    skeletonImage[fluxImage > seuil_flux] = 1
    max_y, max_x = np.unravel_index(np.argmax(skeletonImage),
        skeletonImage.shape)
    mid_x = skeletonImage.shape[1] // 2
    mid_y = skeletonImage.shape[0] // 2
    abscisse_point_rouge = max_x - mid_x
    ordonnee_point_rouge = mid_y - max_y
    angle_pan = math.degrees(math.atan(abscisse_point_rouge / 50))
    angle_tilt = math.degrees(math.atan(ordonnee_point_rouge / 50))
    fig, ax1 = plt.subplots(1, 1, figsize=(10, 10))
    ax1.imshow(skeletonImage, cmap='gray', interpolation='nearest')
    ax1.plot(max_x, max_y, 'ro')
    ax1.axhline(y=mid_y, color='red', linewidth=2)
    ax1.axvline(x=mid_x, color='red', linewidth=2)
    ax1.text(10, 10, f'Abscisse: {abscisse_point_rouge}', color='
red', fontsize=12, backgroundcolor='white')
    ax1.text(10, 30, f'Ordonnee: {ordonnee_point_rouge}', color='
red', fontsize=12, backgroundcolor='white')
    ax1.text(10, 50, f'Angle de pan: {angle_pan:.2f} ', color='red
', fontsize=12, backgroundcolor='white')
    ax1.text(10, 70, f'Angle de tilt: {angle_tilt:.2f} ', color='
red', fontsize=12, backgroundcolor='white')
    ax1.tick_params(axis='both', colors='white')
    plt.show()
    return abscisse_point_rouge, ordonnee_point_rouge, angle_pan,
        angle_tilt

def regler_angle_servo(pi, servo_pin, angle):
    largeur_impulsion = 1500 + (angle / 90.0) * 1000
    pi.set_servo_pulsewidth(servo_pin, largeur_impulsion)

class ApplicationCamera:
    def __init__(self, root, api_key):
        self.root = root
        self.api_key = api_key
        self.camera = PiCamera()
        self.camera.resolution = (640, 480)
        self.raw_capture = PiRGBArray(self.camera, size=(640, 480))
        self.label = tk.Label(root)
        self.label.grid(row=0, column=0, columnspan=2)
        self.bouton_capture = ttk.Button(root, text="Capture",
            command=self.capture_image)
        self.bouton_capture.grid(row=1, column=0, pady=10)
        self.bouton_quitter = ttk.Button(root, text="Quitter",
            command=root.destroy)
        self.bouton_quitter.grid(row=1, column=1, pady=10)
        self.mettre_a_jour_frame()

    def mettre_a_jour_frame(self):
        self.camera.capture(self.raw_capture, format="rgb",

```

```

        use_video_port=True)
    image = Image.fromarray(self.raw_capture.array)
    self.raw_capture.truncate(0)
    self.photo = ImageTk.PhotoImage(image=image)
    self.label.config(image=self.photo)
    self.root.after(10, self.mettre_a_jour_frame)

def capture_image(self):
    self.camera.capture("captured_image.jpg")
    abscisse_point_rouge, ordonnee_point_rouge, angle_pan,
    angle_tilt = squelettiser_et_traiter("captured_image.
    jpg", self.api_key)
    if abscisse_point_rouge is not None and
    ordonnee_point_rouge is not None:
        messagebox.showinfo("R sultat", f"Coordonn es du
        point non noir le plus haut: ({abscisse_point_rouge
        }, {ordonnee_point_rouge})\n"
                                f"Angle de pan: {
                                angle_pan:.2f} \
                                nAngle de tilt: {
                                angle_tilt:.2f} ")

    pi = pigpio.pi()
    if not pi.connected:
        messagebox.showerror("Erreur", " chec  de la
        connexion au d mon pigpio.")
        return

    servo_pin_pan = 17
    servo_pin_tilt = 18

    regler_angle_servo(pi, servo_pin_pan, -angle_pan)
    regler_angle_servo(pi, servo_pin_tilt, -angle_tilt)

    sleep(1)

    pi.set_servo_pulsewidth(servo_pin_pan, 0)
    pi.set_servo_pulsewidth(servo_pin_tilt, 0)
    pi.stop()
    else:
        messagebox.showerror("Erreur", " chec  du traitement de
        l'image.")

def main(api_key):
    root = tk.Tk()
    root.title("Capture de cam ra Raspberry Pi")
    app = ApplicationCamera(root, api_key)
    root.mainloop()

if __name__ == "__main__":
    api_key = '5EuEztQ4aogoHj2LXnSCeHVT'
    main(api_key)

```